sajibkhansk

UNIVERSITY SQL HANDBOOK

Window Functions, CTEs & Subqueries

Real-world SQL patterns using the University database — from easy to intermediate, with 10 practice questions.

WINDOW FUNCTIONS | CTES | SUBQUERIES

10-table university database • MySQL 8 • Easy to Intermediate

CHAPTER 1

What Are Window Functions?

A window function performs a calculation across a **set of rows related to the current row** — without collapsing them into groups like GROUP BY does.

```
-- Anatomy of a window function

ROW_NUMBER() OVER ( PARTITION BY department_id ORDER BY salary DESC)
```

FUNCTION	PARTITION BY	ORDER BY
ROW_NUMBER()	department_id	salary DESC
What to calculate	Divide rows into groups	Order within the window

`ROW_NUMBER()`
Unique sequential number — no ties

`RANK()`
Rank with gaps after ties

`DENSE_RANK()`
Rank without gaps after ties

`NTILE(n)`
Divide rows into n equal buckets

`LAG() / LEAD()`
Look at previous / next row

`SUM() OVER()`
Running total within a window

KEY DIFFERENCE FROM GROUP BY

GROUP BY collapses many rows into one. Window functions keep every row **and add a new calculated column** alongside it.

WINDOW FUNCTIONS

ROW_NUMBER()

Problem: Rank students by GPA within each department. Assign each student a unique sequential rank.

```sql
SELECT
    s.student_id,
    CONCAT(s.first_name, ' ', s.last_name) AS student_name,
    d.dept_name,
    sg.cumulative_gpa,
    ROW_NUMBER() OVER (
        PARTITION BY s.department_id
        ORDER BY sg.cumulative_gpa DESC
    ) AS dept_rank
FROM students s
JOIN departments d ON s.department_id = d.dept_id
JOIN (
    SELECT student_id, MAX(cumulative_gpa) AS cumulative_gpa
    FROM semester_gpa
    GROUP BY student_id
) sg ON s.student_id = sg.student_id
WHERE s.department_id IS NOT NULL
ORDER BY d.dept_name, dept_rank;
```

RESULT (FIRST 8 ROWS)

STUDENT_ID	STUDENT_NAME	DEPT_NAME	CUMULATIVE_GPA	DEPT_RANK
51	Stephanie Lewis	Biology	3.913	1
48	Kevin Park	Biology	3.775	2
31	Melissa Johnson	Business Administration	3.875	1
34	Christopher White	Business Administration	3.800	2
29	Victoria Chen	Business Administration	3.650	3
30	Brandon Lee	Business Administration	3.150	4
7	Nathan Brooks	Computer Science	3.883	1
4	Fatima Hassan	Computer Science	3.850	2

dept_rank resets to 1 for each new department

WINDOW FUNCTIONS

ROW_NUMBER() — How It Works

1. **PARTITION BY s.department_id** splits students into separate windows — one per department. The rank counter resets to 1 for each new window.
2. **ORDER BY sg.cumulative_gpa DESC** sorts students inside each window so the highest GPA gets rank 1.
3. **ROW_NUMBER()** assigns a unique, sequential integer to every row. Two students with identical GPA in the same department will still get different rank numbers (unlike RANK or DENSE_RANK).
4. The subquery in FROM calculates each student's personal best GPA across all semesters before ranking.

WITHOUT PARTITION BY

Ranks run from 1 to total student count. All students are in one big window.

WITH PARTITION BY DEPT

Ranks reset per department. You can find top-N per group, which GROUP BY cannot do.

LEARNING POINT

To get the top 2 students per department, wrap this query in a CTE or subquery and filter `WHERE dept_rank <= 2`. This is the most common ROW_NUMBER() pattern in real interviews.

USE ROW_NUMBER() WHEN

You need **unique ranks with no ties** — for pagination, deduplication (keep the latest row per group), or selecting exactly N rows per partition.

WINDOW FUNCTIONS

RANK() vs DENSE_RANK()

Problem: Rank all courses by total enrollments. Handle ties so the difference between RANK and DENSE_RANK is visible.

```sql
SELECT
    c.course_code,
    c.course_name,
    COUNT(e.enrollment_id)  AS enrollments,
    RANK() OVER (
        ORDER BY COUNT(e.enrollment_id) DESC
    ) AS rnk,
    DENSE_RANK() OVER (
        ORDER BY COUNT(e.enrollment_id) DESC
    ) AS dense_rnk
FROM courses c
JOIN course_sections cs ON c.course_id = cs.course_id
JOIN enrollments e      ON cs.section_id = e.section_id
GROUP BY c.course_id, c.course_code, c.course_name
ORDER BY enrollments DESC
LIMIT 10;
```

RESULT (TOP 10 COURSES)

COURSE_CODE	COURSE_NAME	ENROLLMENTS	RNK	DENSE_RNK
BUS101	Introduction to Business	23	1	1
MATH101	Calculus I	19	2	2
DS201	Python for Data Science	18	3	3
CS101	Introduction to Programming	16	4	4
ECON101	Microeconomics	15	5	5
PHYS101	General Physics I	12	6	6
BIO101	General Biology	12	6	6
CS201	Data Structures & Algorithms	10	8	7
ENG101	Academic Writing & Composition	10	8	7
MATH201	Calculus II	9	10	8

PHYS101 and BIO101 both have 12 enrollments — they share rank 6. After that, RANK skips to 8 while DENSE_RANK continues at 7.

WINDOW FUNCTIONS

RANK vs DENSE_RANK — How They Differ

RANK() — LEAVES GAPS

```
Physics → rank 6
Biology → rank 6
rank 7 is skipped
CS201 → rank 8
```

Uses **position**: if two rows tie at position 6, the next rank is 8 (two rows used up positions 6 and 7).

DENSE_RANK() — NO GAPS

```
Physics → rank 6
Biology → rank 6
rank 7 is NOT skipped
CS201 → rank 7
```

Uses **distinct values**: the next rank is always one more than the previous distinct rank, regardless of how many rows tied.

When to use each

Use RANK()
When you need the true competitive rank — "if 2 students tie for 1st, the next student is 3rd" like a sports leaderboard.

Use DENSE_RANK()
When you want contiguous tier labels — like "Top 3 tiers" — where gaps in the numbers would confuse the audience.

ROW_NUMBER VS RANK VS DENSE_RANK IN ONE SENTENCE

ROW_NUMBER: unique always. RANK: ties share a rank, next rank skips. DENSE_RANK: ties share a rank, next rank is always +1.

WINDOW FUNCTIONS

NTILE(4) — Dividing into Quartiles

Problem: Divide all students into 4 equal GPA quartiles. Quartile 1 = highest achievers.

```sql
SELECT
    CONCAT(s.first_name, ' ', s.last_name) AS student_name,
    d.dept_name,
    sg.cumulative_gpa,
    NTILE(4) OVER (
        ORDER BY sg.cumulative_gpa DESC
    ) AS quartile
FROM students s
JOIN departments d ON s.department_id = d.dept_id
JOIN (
    SELECT student_id, MAX(cumulative_gpa) AS cumulative_gpa
    FROM semester_gpa GROUP BY student_id
) sg ON s.student_id = sg.student_id
WHERE s.department_id IS NOT NULL
ORDER BY cumulative_gpa DESC
LIMIT 12;
```

RESULT (TOP 12 SHOWN)

STUDENT_NAME	DEPT_NAME	CUMULATIVE_GPA	QUARTILE
Benjamin Moore	Mathematics	3.975	1
Cassandra Green	English Literature	3.963	1
Ryan Nelson	Economics	3.938	1
Stephanie Lewis	Biology	3.913	1
Marcus Stone	Physics	3.900	1
Nathan Brooks	Computer Science	3.883	1
Laila Ahmed	Physics	3.875	1
Melissa Johnson	Business Administration	3.875	1
Jasmine Washington	Data Science	3.850	1
Priya Nair	Computer Science	3.850	1
Fatima Hassan	Computer Science	3.850	1
Christopher White	Business Administration	3.800	2

Quartile 1 = top 25% of students by GPA

HOW NTILE WORKS

NTILE(4) divides all rows (after ORDER BY) into 4 buckets as evenly as possible. If rows don't divide evenly, earlier buckets get the extra rows. Use NTILE(100) for percentiles.

WINDOW FUNCTIONS

LAG() and LEAD()

Problem: For each student, compare their current semester GPA to the previous semester (LAG) and peek at the next semester (LEAD).

```sql
SELECT
    CONCAT(s.first_name, ' ', s.last_name) AS student_name,
    sg.year,  sg.semester,  sg.cumulative_gpa,
    LAG(sg.cumulative_gpa)  OVER (
        PARTITION BY s.student_id
        ORDER BY sg.year, sg.semester
    ) AS prev_gpa,
    LEAD(sg.cumulative_gpa) OVER (
        PARTITION BY s.student_id
        ORDER BY sg.year, sg.semester
    ) AS next_gpa,
    ROUND(sg.cumulative_gpa - LAG(sg.cumulative_gpa) OVER (
        PARTITION BY s.student_id
        ORDER BY sg.year, sg.semester
    ), 3) AS gpa_delta
FROM semester_gpa sg
JOIN students s ON sg.student_id = s.student_id
WHERE s.student_id IN (1, 2, 3)
ORDER BY s.student_id, sg.year, sg.semester;
```

RESULT

STUDENT_NAME	YEAR	SEMESTER	CUMULATIVE_GPA	PREV_GPA	NEXT_GPA	GPA_DELTA
Alex Turner	2022	Fall	3.450	*NULL*	3.458	*NULL*
Alex Turner	2023	Fall	3.458	3.450	3.533	0.008
Alex Turner	2023	Spring	3.533	3.458	*NULL*	0.075
Mei Lin	2021	Fall	3.700	*NULL*	3.700	*NULL*
Mei Lin	2022	Fall	3.700	3.700	3.750	0.000
Mei Lin	2022	Spring	3.750	3.700	3.740	0.050
Mei Lin	2023	Fall	3.740	3.750	3.713	-0.010
Mei Lin	2023	Spring	3.713	3.740	*NULL*	-0.027
Jordan Smith	2023	Fall	3.100	*NULL*	*NULL*	*NULL*

NULL = no previous/next semester in the dataset

WINDOW FUNCTIONS

LAG / LEAD — How They Work

1. **PARTITION BY s.student_id** creates a separate window for each student. LAG and LEAD only look at rows within the same partition — they never peek into another student's rows.
2. **ORDER BY sg.year, sg.semester** defines the chronological order within each student's window. LAG looks one row backward in that order; LEAD looks one row forward.
3. The **first row** in each partition has no previous row, so LAG returns NULL. The **last row** in each partition has no next row, so LEAD returns NULL.
4. **gpa_delta** = current_gpa minus prev_gpa. A positive delta means improvement; negative means decline.

`LAG(col, n, default)`
Look back n rows. Default value when NULL. Default n=1, default value=NULL.

`LEAD(col, n, default)`
Look forward n rows. Same optional parameters. Useful for forecasting or next-period comparison.

REAL-WORLD USE

LAG is used in finance to calculate day-over-day stock price change. LEAD is used to flag customers who will churn next month. Both are essential for time-series analysis in SQL.

WATCH OUT

Mixing up ORDER BY inside OVER() with ORDER BY at the query level is a common mistake. The ORDER BY inside OVER() controls the window frame only — the final result order needs its own ORDER BY at the end.

WINDOW FUNCTIONS

FIRST_VALUE() + AVG() OVER()

Problem: Show every professor alongside their department average salary and the name of the highest-paid person in their department.

```sql
SELECT
    CONCAT(p.first_name, ' ', p.last_name) AS professor_name,
    d.dept_name,  p.salary,
    ROUND(AVG(p.salary) OVER (
        PARTITION BY p.department_id
    )) AS dept_avg_salary,
    FIRST_VALUE(CONCAT(p.first_name, ' ', p.last_name)) OVER (
        PARTITION BY p.department_id ORDER BY p.salary DESC
    ) AS highest_paid,
    ROUND(p.salary - AVG(p.salary) OVER (
        PARTITION BY p.department_id
    ), 2) AS vs_avg
FROM professors p
JOIN departments d ON p.department_id = d.dept_id
ORDER BY d.dept_name, p.salary DESC
LIMIT 10;
```

RESULT (FIRST 10 ROWS)

PROFESSOR_NAME	DEPT_NAME	SALARY	DEPT_AVG_SALARY	HIGHEST_PAID	VS_AVG
Lisa Chen	Biology	132000.00	114000	Lisa Chen	18000.00
Andrew Wilson	Biology	96000.00	114000	Lisa Chen	-18000.00
Emily Rodriguez	Business Admin	145000.00	118333	Emily Rodriguez	26667.00
Kevin Zhang	Business Admin	118000.00	118333	Emily Rodriguez	-333.00
Nicole Turner	Business Admin	92000.00	118333	Emily Rodriguez	-26333.00
Robert Chen	Computer Science	135000.00	103667	Robert Chen	31333.00
Jennifer Park	Computer Science	98000.00	103667	Robert Chen	-5667.00
Marcus Johnson	Computer Science	78000.00	103667	Robert Chen	-25667.00
Aisha Patel	Data Science	142000.00	103333	Aisha Patel	38667.00
Carlos Rivera	Data Science	100000.00	103333	Aisha Patel	-3333.00

FIRST_VALUE

Always returns the first row's value in the ordered partition — here, the highest-paid professor name repeats on every row in that department.

WINDOW FUNCTIONS

SUM() OVER() — Running Total

Problem: For each student, show a running total of scholarship money received, ordered by year.

```sql
SELECT
    CONCAT(s.first_name, ' ', s.last_name) AS student_name,
    sc.scholarship_name,
    ss.academic_year,
    ss.amount_awarded,
    SUM(ss.amount_awarded) OVER (
        PARTITION BY ss.student_id
        ORDER BY ss.academic_year, sc.scholarship_name
    ) AS running_total
FROM student_scholarships ss
JOIN students s    ON ss.student_id    = s.student_id
JOIN scholarships sc ON ss.scholarship_id = sc.scholarship_id
WHERE ss.student_id IN (2, 4, 6, 7, 10, 11, 12)
ORDER BY ss.student_id, ss.academic_year
LIMIT 10;
```

RESULT (FIRST 10 ROWS)

STUDENT_NAME	SCHOLARSHIP_NAME	ACADEMIC_YEAR	AMOUNT_AWARDED	RUNNING_TOTAL
Mei Lin	Dean's Excellence Award	2023-24	5000.00	5000.00
Mei Lin	CS Innovation Grant	2023-24	4000.00	9000.00
Fatima Hassan	STEM Merit Scholarship	2023-24	3500.00	3500.00
Fatima Hassan	CS Innovation Grant	2023-24	4000.00	7500.00
Isabella Garcia	CS Innovation Grant	2023-24	4000.00	4000.00
Nathan Brooks	Dean's Excellence Award	2022-23	5000.00	5000.00
Nathan Brooks	CS Innovation Grant	2022-23	4000.00	9000.00
Priya Nair	STEM Merit Scholarship	2024-25	3500.00	3500.00
Lucas Martini	STEM Merit Scholarship	2023-24	3500.00	3500.00
Emma Johansson	Mathematics Achievement Award	2023-24	3000.00	3000.00

HOW RUNNING TOTALS WORK

SUM() OVER(PARTITION BY ... ORDER BY ...) accumulates the value from the first row in the partition up to and including the current row. This is called a **cumulative window frame**.

REFERENCE

Window Functions — Cheat Sheet

FUNCTION	WHAT IT RETURNS	NEEDS ORDER BY?	TYPICAL USE
`ROW_NUMBER()`	Unique sequential int — no ties	Yes	Deduplication, top-N per group
`RANK()`	Rank with gaps after ties	Yes	Sports-style leaderboards
`DENSE_RANK()`	Rank without gaps after ties	Yes	Tier labels, percentile bands
`NTILE(n)`	Bucket number 1..n	Yes	Quartiles, deciles
`LAG(col, n)`	Value from n rows behind	Yes	Period-over-period change
`LEAD(col, n)`	Value from n rows ahead	Yes	Next-period forecasting
`FIRST_VALUE(col)`	First value in the window	Yes	Show group leader on every row
`SUM() OVER()`	Cumulative or total sum	Optional	Running totals
`AVG() OVER()`	Avg across the window	Optional	Moving average, vs-avg column
`COUNT() OVER()`	Row count in the window	Optional	Group size alongside each row

GOLDEN RULE

If you need to keep all individual rows **and** also compute a group-level metric, reach for a window function over a subquery — it runs in one pass.

CHAPTER 2

What Are CTEs?

A CTE (Common Table Expression) is a **named temporary result set** defined at the top of a query with the `WITH` keyword. It acts like a virtual table you can reference in the main SELECT.

```
-- Anatomy of a CTE
WITH cte_name AS (
    -- Any valid SELECT goes here
    SELECT ... FROM ...
)
SELECT * FROM cte_name;
```

Simple CTE
One named sub-result. Cleaner than a subquery in FROM.

Multi-CTE
Multiple WITH clauses separated by commas. CTEs can reference earlier ones.

Recursive CTE
WITH RECURSIVE — CTE references itself. Used for trees, hierarchies, and sequences.

CTE VS SUBQUERY

A CTE names the logic once and reuses it anywhere in the query. A subquery repeats the logic each time it appears. For complex queries, CTEs are dramatically easier to read and debug.

CTE VS TEMP TABLE

CTEs live only for the duration of a single statement. Temp tables persist in the session. For multi-statement workflows use temp tables; for one complex query, prefer CTEs.

CTES

Simple CTE

Problem: Find professors who earn above their department average — and show by how much.

```sql
WITH dept_avg AS (
    SELECT department_id,
           ROUND(AVG(salary)) AS avg_salary
    FROM professors
    GROUP BY department_id
)
SELECT
    CONCAT(p.first_name, ' ', p.last_name) AS professor_name,
    d.dept_name,
    p.salary,
    da.avg_salary      AS dept_avg,
    ROUND(p.salary - da.avg_salary, 2) AS above_avg_by
FROM professors p
JOIN dept_avg   da ON p.department_id = da.department_id
JOIN departments d ON p.department_id = d.dept_id
WHERE p.salary > da.avg_salary
ORDER BY above_avg_by DESC;
```

RESULT

PROFESSOR_NAME	DEPT_NAME	SALARY	DEPT_AVG	ABOVE_AVG_BY
Aisha Patel	Data Science	142000.00	103333	38667.00
David Kim	Economics	138000.00	106500	31500.00
Robert Chen	Computer Science	135000.00	103667	31333.00
Sarah Williams	Mathematics	128000.00	101000	27000.00
Emily Rodriguez	Business Administration	145000.00	118333	26667.00
James Morrison	Physics	140000.00	117500	22500.00
Michael Thompson	English Literature	115000.00	93500	21500.00
Lisa Chen	Biology	132000.00	114000	18000.00

HOW IT WORKS

The CTE `dept_avg` calculates one average salary per department. The main SELECT joins professors back to that CTE, then filters WHERE salary > dept average. Without the CTE you'd need a correlated subquery in the WHERE clause — much harder to read.

CTES

Multi-CTE

Problem: Find students with a GPA above 3.85, and also show how many courses each of them is enrolled in — combining two independent calculations.

```sql
WITH top_students AS (
    SELECT student_id, MAX(cumulative_gpa) AS cumulative_gpa
    FROM semester_gpa
    GROUP BY student_id
    HAVING MAX(cumulative_gpa) >= 3.85
),
enrollment_count AS (
    SELECT student_id, COUNT(DISTINCT section_id) AS total_courses
    FROM enrollments
    GROUP BY student_id
)
SELECT
    CONCAT(s.first_name, ' ', s.last_name) AS student_name,
    d.dept_name,
    ts.cumulative_gpa,
    ec.total_courses
FROM top_students   ts
JOIN students       s  ON ts.student_id = s.student_id
JOIN departments    d  ON s.department_id = d.dept_id
JOIN enrollment_count ec ON ts.student_id = ec.student_id
ORDER BY ts.cumulative_gpa DESC;
```

RESULT

STUDENT_NAME	DEPT_NAME	CUMULATIVE_GPA	TOTAL_COURSES
Benjamin Moore	Mathematics	3.975	3
Cassandra Green	English Literature	3.963	3
Ryan Nelson	Economics	3.938	3
Stephanie Lewis	Biology	3.913	3
Marcus Stone	Physics	3.900	3
Nathan Brooks	Computer Science	3.883	4
Laila Ahmed	Physics	3.875	3
Melissa Johnson	Business Administration	3.875	3

CTES

Multi-CTE — How It Works

1. **top_students** — the first CTE uses HAVING to keep only students whose personal best GPA (MAX across all semesters) is at least 3.85. This scans semester_gpa once.
2. **enrollment_count** — the second CTE counts distinct section enrollments per student. This is a completely independent scan of the enrollments table.
3. The **main SELECT** joins both CTEs together on student_id, then enriches with the students and departments tables for human-readable names.

WITHOUT CTES

You'd need two subqueries in the FROM clause. Nesting them makes the logic hard to follow and difficult to debug or modify later.

WITH MULTI-CTE

Each CTE is named, isolated, and readable on its own. You can test each CTE independently by running just its SELECT. This is the key advantage.

MULTI-CTE RULE

CTEs are defined in order — a later CTE can reference an earlier one, but not the other way around. Each CTE is separated by a comma inside the WITH block.

PERFORMANCE NOTE

In MySQL 8, CTEs are not automatically materialized or cached — the optimizer may inline them. For heavy CTEs that are referenced multiple times, a temporary table may perform better.

CTES

Recursive CTE

Problem: Walk the professor management hierarchy — show each professor indented under their department head.

```
WITH RECURSIVE org_tree AS (
    -- Anchor: department heads (no manager)
    SELECT
        professor_id,
        CONCAT(first_name, ' ', last_name) AS name,
        title, 0 AS lvl,
        CONCAT(first_name, ' ', last_name) AS org_chart
    FROM professors WHERE manager_id IS NULL

    UNION ALL

    -- Recursive step: each direct report
    SELECT
        p.professor_id,
        CONCAT(p.first_name, ' ', p.last_name),
        p.title,  ot.lvl + 1,
        CONCAT(REPEAT('  ', ot.lvl + 1),
               p.first_name, ' ', p.last_name)
    FROM professors p
    JOIN org_tree ot ON p.manager_id = ot.professor_id
)
SELECT org_chart, title, lvl AS level
FROM org_tree ORDER BY org_chart LIMIT 12;
```

RESULT (ORG CHART)

ORG_CHART	TITLE	LEVEL
Aisha Patel	Department Head	0
Carlos Rivera	Associate Professor	1
Hannah White	Lecturer	1
David Kim	Department Head	0
Priya Sharma	Assistant Professor	1
Emily Rodriguez	Department Head	0
Kevin Zhang	Professor	1
Nicole Turner	Associate Professor	1
James Morrison	Department Head	0
Rachel Adams	Associate Professor	1
Lisa Chen	Department Head	0
Andrew Wilson	Associate Professor	1

Indentation shows hierarchy level (2 spaces per level)

CTES

Recursive CTE — How It Works

1. **Anchor member** — the first SELECT (before UNION ALL) runs once and produces the starting rows. Here it selects professors with no manager (department heads, where manager_id IS NULL).
2. **Recursive member** — the second SELECT (after UNION ALL) joins the professors table back to the CTE itself (`org_tree`). It finds every professor whose manager is already in the result set.
3. MySQL keeps re-running the recursive member, adding new rows each time, until no new rows are produced. The `lvl` counter increments with each step.
4. **CONCAT(REPEAT(' ', lvl+1), name)** adds two spaces of indentation per level, creating the visual tree in the org_chart column.

`Iteration 0 (anchor)`
Department heads — managers with no manager themselves. Level = 0.

`Iteration 1 (recursive)`
Direct reports of those department heads. Level = 1.

SAFETY VALVE

Always include a LIMIT clause or a depth counter (level < N) in recursive CTEs to prevent infinite loops if the data contains cycles.

REAL-WORLD USES

Recursive CTEs are used for org charts, file system trees, bill of materials (BOM), category hierarchies, and route-planning queries.

CHAPTER 3

What Are Subqueries?

A subquery is a **SELECT statement nested inside another SQL statement**. It runs first and hands its result to the outer query.

SCALAR SUBQUERY

Returns a single value. Used in SELECT or WHERE to compute one number per outer row.

IN / NOT IN

Returns a list of values. The outer WHERE checks membership in that list.

CORRELATED SUBQUERY

References the outer query's table. Runs once per outer row — flexible but can be slow on large datasets.

EXISTS

Returns TRUE if the subquery produces any row. Stops as soon as one match is found — very efficient.

SUBQUERY VS CTE VS JOIN

Use a subquery when the logic is short and used once. Use a CTE when you need to name it or reuse it. Use a JOIN when you need columns from both sides. Mixing them strategically is what separates intermediate from advanced SQL.

SUBQUERIES

Scalar Subquery

Problem: Show each professor's salary alongside their department average — computed as a scalar subquery in SELECT.

```sql
SELECT
    CONCAT(p.first_name, ' ', p.last_name) AS professor_name,
    d.dept_name,  p.salary,
    (SELECT ROUND(AVG(salary))
     FROM professors p2
     WHERE p2.department_id = p.department_id) AS dept_avg,
    ROUND(p.salary - (
        SELECT AVG(salary) FROM professors p2
        WHERE p2.department_id = p.department_id
    ), 2) AS vs_avg
FROM professors p
JOIN departments d ON p.department_id = d.dept_id
ORDER BY d.dept_name, p.salary DESC
LIMIT 8;
```

RESULT (FIRST 8 ROWS)

PROFESSOR_NAME	DEPT_NAME	SALARY	DEPT_AVG	VS_AVG
Lisa Chen	Biology	132000.00	114000	18000.00
Andrew Wilson	Biology	96000.00	114000	-18000.00
Emily Rodriguez	Business Admin	145000.00	118333	26667.00
Kevin Zhang	Business Admin	118000.00	118333	-333.00
Nicole Turner	Business Admin	92000.00	118333	-26333.00
Robert Chen	Computer Science	135000.00	103667	31333.00
Jennifer Park	Computer Science	98000.00	103667	-5667.00
Marcus Johnson	Computer Science	78000.00	103667	-25667.00

HOW IT WORKS

The scalar subquery in SELECT runs once per outer row, using `p.department_id` from the outer query. It returns exactly one value. The same subquery appears again in the vs_avg calculation — this is repetitive; a CTE or window function would be cleaner.

WATCH OUT

A scalar subquery must return exactly one row and one column. If it returns more than one row, MySQL throws an error.

SUBQUERIES

IN and NOT IN Subqueries

NOT IN — Students never enrolled

```sql
-- Students with NO enrollments
SELECT
    student_id,
    CONCAT(first_name, ' ', last_name) AS student_name,
    status, department_id
FROM students
WHERE student_id NOT IN (
    SELECT DISTINCT student_id FROM enrollments
)
LIMIT 5;
```

RESULT

STUDENT_ID	STUDENT_NAME	STATUS	DEPARTMENT_ID
73	Chris Morgan	Active	*NULL*
74	Taylor Swift	Active	*NULL*
75	Casey Jordan	Active	*NULL*
76	Sam Patel	Active	*NULL*
77	Alex Kim	Active	*NULL*

IN — Students with a scholarship

```sql
-- Students who HAVE a scholarship
SELECT
    student_id,
    CONCAT(first_name, ' ', last_name) AS student_nam
    status
FROM students
WHERE student_id IN (
    SELECT DISTINCT student_id FROM student_scholarsh
)
LIMIT 5;
```

RESULT

STUDENT_ID	STUDENT_NAME	STATUS
1	Alex Turner	Active
2	Mei Lin	Active
3	Jordan Smith	Active
7	Nathan Brooks	Graduated
9	Ethan Clark	Active

NOT IN AND NULLS

If the subquery result contains any NULL value, NOT IN returns no rows at all. Always add WHERE col IS NOT NULL inside the subquery when using NOT IN.

SUBQUERIES

Correlated Subquery

Problem: Find students whose GPA is above their department average — compare each student to their own department's average.

```sql
SELECT
    CONCAT(s.first_name, ' ', s.last_name) AS student_name,
    d.dept_name,
    sg.cumulative_gpa,
    ROUND((
        SELECT AVG(sg2.cumulative_gpa)
        FROM semester_gpa sg2
        JOIN students s2 ON sg2.student_id = s2.student_id
        WHERE s2.department_id = s.department_id
    ), 3) AS dept_avg
FROM students s
JOIN departments d   ON s.department_id = d.dept_id
JOIN semester_gpa sg ON s.student_id = sg.student_id
WHERE sg.cumulative_gpa > (
    SELECT AVG(sg2.cumulative_gpa)
    FROM semester_gpa sg2
    JOIN students s2 ON sg2.student_id = s2.student_id
    WHERE s2.department_id = s.department_id
)
GROUP BY s.student_id, d.dept_name, sg.cumulative_gpa
ORDER BY d.dept_name, sg.cumulative_gpa DESC
LIMIT 8;
```

RESULT (FIRST 8 ROWS)

STUDENT_NAME	DEPT_NAME	CUMULATIVE_GPA	DEPT_AVG
Stephanie Lewis	Biology	3.913	3.844
Melissa Johnson	Business Administration	3.875	3.404
Christopher White	Business Administration	3.800	3.404
Victoria Chen	Business Administration	3.650	3.404
Nathan Brooks	Computer Science	3.883	3.475
Priya Nair	Computer Science	3.850	3.475
Fatima Hassan	Computer Science	3.850	3.475
Mei Lin	Computer Science	3.713	3.475

WHAT MAKES IT CORRELATED

The inner subquery uses `s.department_id` from the outer SELECT. This ties the subquery to the outer row — it re-executes for every student, each time calculating that student's specific department average.

PERFORMANCE

Correlated subqueries can be slow on large tables because they run once per outer row. A window function (AVG OVER PARTITION BY) or a CTE with a JOIN usually performs the same logic faster.

SUBQUERIES

EXISTS Subquery

Problem: Find departments that have at least one professor earning over $130,000 — list the department with professor headcount.

```sql
SELECT
    d.dept_name,
    d.building,
    COUNT(p.professor_id) AS professors
FROM departments d
JOIN professors p ON d.dept_id = p.department_id
WHERE EXISTS (
    SELECT 1 FROM professors p2
    WHERE p2.department_id = d.dept_id
    AND p2.salary > 130000
)
GROUP BY d.dept_id, d.dept_name, d.building
ORDER BY professors DESC;
```

RESULT

DEPT_NAME	BUILDING	PROFESSORS
Computer Science	Engineering Building	3
Mathematics	Science Hall	3

EXISTS vs IN — when to use which

Use EXISTS when
You only care whether a match exists — not the actual values. EXISTS stops at the first match, making it fast even on huge tables.

Use IN when
You have a small list of specific values to match against. Less efficient when the subquery returns many rows.

EXISTS WITH NOT EXISTS

NOT EXISTS finds rows where no match exists in the subquery — the opposite of EXISTS. It avoids the NULL pitfall that NOT IN has.

REFERENCE

When To Use What

GOAL	BEST TOOL	WHY
Rank rows within groups	Window Function (RANK/ROW_NUMBER)	Keeps all rows, adds rank column
Compare row to previous period	Window Function (LAG/LEAD)	Looks back/forward within a partition
Running total or moving avg	Window Function (SUM/AVG OVER)	Cumulative frame within partition
Named reusable sub-result	CTE (WITH)	Clean, testable, readable
Hierarchy / tree traversal	Recursive CTE	Only way to walk unlimited depth
Compute one value per outer row	Scalar Subquery or Window Fn	Window fn usually faster
Filter: value in a list	IN subquery	Simple membership test
Filter: no match in set	NOT EXISTS (prefer over NOT IN)	Safe with NULLs, stops at first match
Filter: at least one match exists	EXISTS subquery	Stops early, very efficient
Complex multi-step logic	Multi-CTE	Each step named and debuggable

THE MOST VERSATILE COMBO

Multi-CTE + Window Functions: use CTEs to build each sub-result cleanly, then use window functions inside or on top of them for ranking, running totals, and comparison columns.

PRACTICE

10 Practice Questions

Apply what you learned. Each question has a short description and the expected result table — figure out the SQL on your own.

Q1 WINDOW FN
Rank professors by salary within each department — show top 2 per department.

Q2 CTE
Find rooms that have never been assigned to any course section.

Q3 IN
List students who have received at least one scholarship (show 8 rows).

Q4 NOT IN
Find professors who have never taught a course section.

Q5 LAG
Find students whose GPA dropped compared to their previous semester.

Q6 MULTI-CTE
For each department, compare the average scholarship amount to the university overall average.

Q7 NTILE
Divide all professors into 3 salary tiers using NTILE: High, Mid, Low.

Q8 EXISTS
Find students who are enrolled in a course AND also have a scholarship.

Q9 CORRELATED
For each department, find the most expensive scholarship any of its students received.

Q10 MULTI-CTE + WF
Student performance report: latest GPA, GPA rank within department, total scholarships earned.

PRACTICE

Q1 WINDOW FN

Rank professors by salary within each department. Show only the top 2 per department.

EXPECTED RESULT (TOP 10 SHOWN)

PROFESSOR_NAME	DEPT_NAME	SALARY	SALARY_RANK
Lisa Chen	Biology	132000.00	1
Andrew Wilson	Biology	96000.00	2
Emily Rodriguez	Business Admin	145000.00	1
Kevin Zhang	Business Admin	118000.00	2
Robert Chen	Computer Science	135000.00	1
Jennifer Park	Computer Science	98000.00	2
Aisha Patel	Data Science	142000.00	1
Carlos Rivera	Data Science	100000.00	2
David Kim	Economics	138000.00	1
Priya Sharma	Economics	75000.00	2

16 rows total — 2 per department across 8 depts

HINT

Use ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary DESC). Wrap it in a CTE or subquery, then filter WHERE salary_rank <= 2.

Q2 CTE

Find rooms that have never been used for any course section.

EXPECTED RESULT

ROOM_ID	BUILDING	ROOM_NUMBER	ROOM_TYPE	CAPACITY
15	Library	LIB-101	Seminar Room	20

1 room never assigned to any section

HINT

Use a CTE to get the list of room_ids that appear in course_sections, then select rooms WHERE room_id NOT IN (that CTE) — or use NOT EXISTS.

PRACTICE

Q3 IN

List students who have received at least one scholarship (show 8 rows, ordered by student_id).

EXPECTED RESULT

STUDENT_ID	STUDENT_NAME	STATUS
1	Alex Turner	Active
2	Mei Lin	Active
3	Jordan Smith	Active
4	Fatima Hassan	Active
6	Isabella Garcia	Active
7	Nathan Brooks	Graduated
9	Ethan Clark	Active
10	Priya Nair	Active

8 rows shown, more exist

HINT

Use WHERE student_id IN (SELECT DISTINCT student_id FROM student_scholarships).

Q4 NOT IN

Find professors who have never taught any course section.

EXPECTED RESULT

PROFESSOR_NAME	TITLE	DEPT_NAME
Hannah White	Lecturer	Data Science

Only 1 professor has never been assigned to a section

HINT

Use WHERE professor_id NOT IN (SELECT DISTINCT professor_id FROM course_sections WHERE professor_id IS NOT NULL). Always add IS NOT NULL inside NOT IN subqueries.

PRACTICE

Q5 LAG

Find students whose cumulative GPA dropped compared to their previous semester entry.

EXPECTED RESULT

STUDENT_NAME	YEAR	SEMESTER	PREV_GPA	CURRENT_GPA	GPA_DROP
Ethan Clark	2023	Fall	3.200	3.100	-0.100
James Foster	2023	Fall	3.000	2.900	-0.100
Samuel Torres	2023	Fall	3.200	3.117	-0.083
Emma Johansson	2022	Spring	3.367	3.300	-0.067
Yuki Tanaka	2023	Spring	3.267	3.200	-0.067
Tyler Reed	2023	Spring	3.733	3.675	-0.058
Nathan Brooks	2022	Fall	3.950	3.900	-0.050

HINT

Use LAG(cumulative_gpa) OVER(PARTITION BY student_id ORDER BY year, semester). Filter WHERE current_gpa < prev_gpa.

Q6 MULTI-CTE

For each department, compare the average schola the university-wide average.

EXPECTED RESULT

DEPT_NAME	AVG_SCHOLARSHIP	OVERALL_AVG
Physics	4500	335
Computer Science	4000	335
Biology	3500	335
Mathematics	3000	335
Business Admin	3000	335
Economics	3000	335
English Literature	2500	335

HINT

Use two CTEs: one for dept averages (JOIN schol student_scholarships to students), one for the o university average. Join them in the final SELECT.

PRACTICE

Q7 NTILE

Divide professors into 3 salary tiers using NTILE: tier 1 = High, tier 2 = Mid, tier 3 = Low.

EXPECTED RESULT (10 OF 20 SHOWN)

PROFESSOR_NAME	DEPT_NAME	SALARY	SALARY_TIER
Lisa Chen	Biology	132000.00	Mid
Andrew Wilson	Biology	96000.00	Low
Emily Rodriguez	Business Admin	145000.00	High
Kevin Zhang	Business Admin	118000.00	Mid
Nicole Turner	Business Admin	92000.00	Low
Robert Chen	Computer Science	135000.00	High
Jennifer Park	Computer Science	98000.00	Mid
Marcus Johnson	Computer Science	78000.00	Low
Aisha Patel	Data Science	142000.00	High
Carlos Rivera	Data Science	100000.00	Mid

20 professors total across all departments

HINT

Use NTILE(3) OVER(ORDER BY salary DESC). Wrap in a CTE, then CASE WHEN tier=1 THEN 'High' ... in the outer SELECT.

Q8 EXISTS

Find students who are both enrolled in a course section AND hold at least one scholarship.

EXPECTED RESULT (FIRST 8 SHOWN)

STUDENT_ID	STUDENT_NAME	DEPT_NAME
48	Kevin Park	Biology
50	Isaac Robinson	Biology
51	Stephanie Lewis	Biology
29	Victoria Chen	Business Administration
31	Melissa Johnson	Business Administration
34	Christopher White	Business Administration
36	Matthew Davis	Business Administration
2	Mei Lin	Computer Science

More rows exist across other departments

HINT

Use WHERE EXISTS (SELECT 1 FROM enrollments WHERE student_id = s.student_id) AND EXISTS (SELECT 1 FROM student_scholarships WHERE student_id = s.student_id).

PRACTICE

Q9 CORRELATED

For each department, find the most expensive scholarship that any of its students received.

EXPECTED RESULT

DEPT_NAME	SCHOLARSHIP_NAME	AMOUNT
Physics	Physics Research Fellowship	4500.00
Computer Science	CS Innovation Grant	4000.00
Biology	Biological Sciences Grant	3500.00
Mathematics	Mathematics Achievement Award	3000.00
Business Admin	Business Leadership Scholarship	3000.00
Economics	Economics Policy Award	3000.00
English Literature	Humanities Excellence Award	2500.00

HINT

Use a correlated subquery in WHERE: WHERE sc.amount = (SELECT MAX(sc2.amount) FROM scholarships sc2 JOIN student_scholarships ss2 ON ... JOIN students s2 ON ... WHERE s2.department_id = d.dept_id).

Q10 MULTI-CTE + WF

Student performance report: latest cumulative GPA, GPA rank within department, a total scholarship money received.

EXPECTED RESULT (FIRST 10 SHOWN)

STUDENT_NAME	DEPT_NAME	LATEST_GPA	DEPT_GPA_RANK	SCHOLARSHIP_
Stephanie Lewis	Biology	3.913	1	85
Kevin Park	Biology	3.775	2	35
Melissa Johnson	Business Admin	3.875	1	50
Christopher White	Business Admin	3.800	2	80
Victoria Chen	Business Admin	3.650	3	30
David Nguyen	Business Admin	3.150	4	
Brandon Lee	Business Admin	3.150	4	
Ashley Brown	Business Admin	2.800	6	
Nathan Brooks	Computer Science	3.883	1	90
Fatima Hassan	Computer Science	3.850	2	75

More rows exist for remaining departments

HINT

Use 3 CTEs: latest GPA per student (MAX), scholarship totals (SUM with COALESCE zeros), GPA rank (ROW_NUMBER OVER PARTITION BY dept). Join all three in the fina SELECT.

YOU NOW KNOW HOW TO USE

ROW_NUMBER • RANK • DENSE_RANK

NTILE • LAG • LEAD

FIRST_VALUE • SUM OVER • AVG OVER

Simple CTE • Multi-CTE • Recursive CTE

Scalar • IN • NOT IN • Correlated • EXISTS

Keep practising on real data.

The difference between knowing and fluency is writing these queries until the patterns feel natural. Try combining CTEs with window functions — that is where intermediate SQL becomes advanced.

More SQL content on LinkedIn

linkedin.com/in/sajibkhansk